

Università degli Studi di Milano

Studio e Testing dell'attacco CRIME a TLS

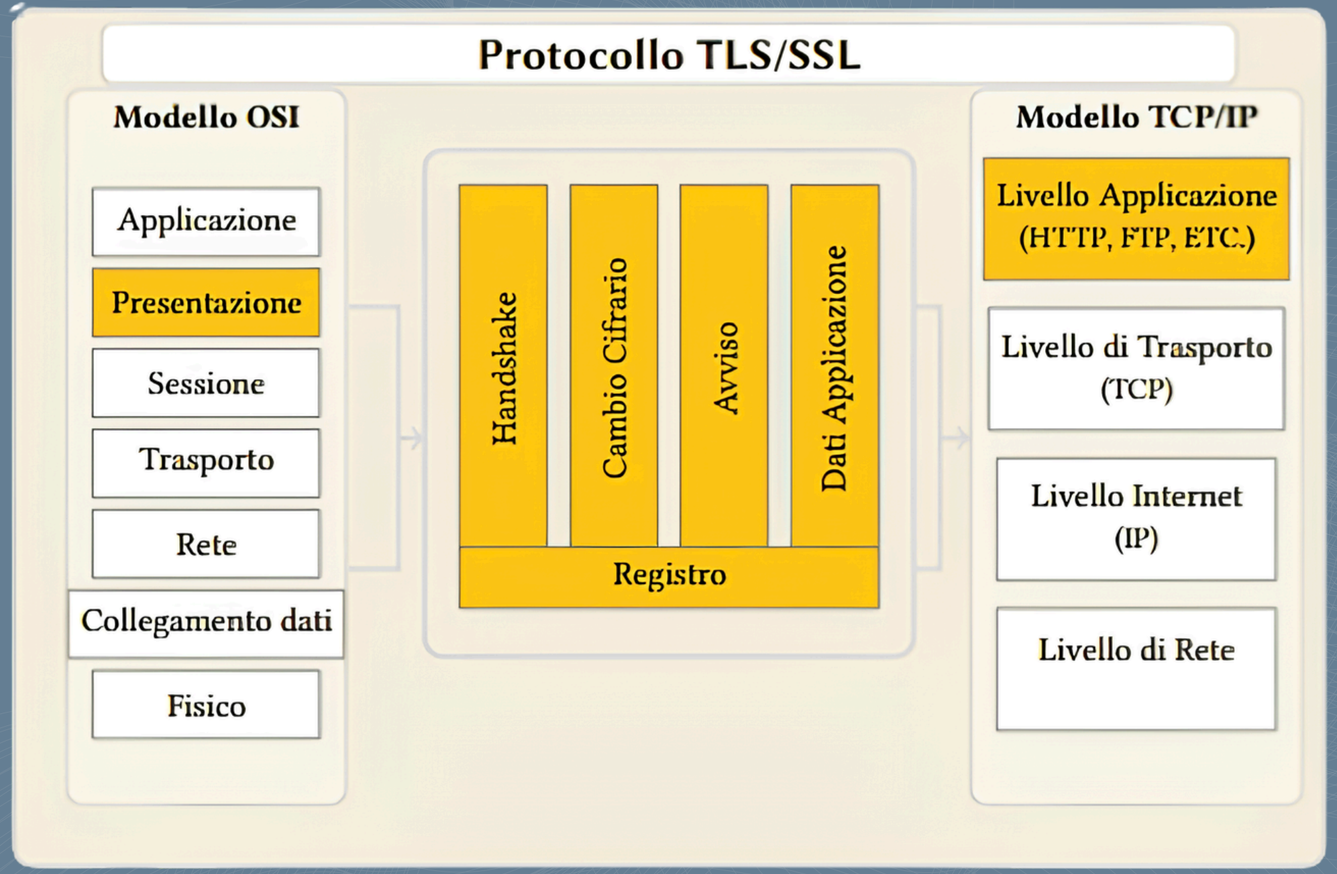
Candidata: Michela Francesca Firrera
Matricola: 02022A
Relatore: Andrea Visconti

Obiettivi

- Analizzare l'attacco CRIME
- Comprendere come compressione e cifratura, combinate, creino un canale di attacco
- Dimostrare il funzionamento dell'attacco attraverso una serie di esperimenti progressivi

TLS

- Autenticazione
- Integrità
- Riservatezza



TLS

Architettura Interna

- Handshake Protocol
- Change Cipher Spec
- Alert Protocol
- Application Data Protocol

Handshake TLS 1.2

- 2 round trip (2-RTT) per completare la negoziazione
- Scambio sequenziale di certificati e chiavi
- Attivazione finale della cifratura con Change Cipher Spec

Handshake TLS 1.3

- Ridotto a 1 solo round trip (1-RTT)
- Il client invia subito la propria Key Share nel ClientHello
- Supporto a 0-RTT per sessioni già note

TLS

Compressione

- L'algoritmo DEFLATE elimina le ridondanze sostituendo sequenze ripetute con puntatori
- Più ripetizioni nel payload → pacchetto più piccolo
- La compressione avviene prima della cifratura

→

Attenzione

- La cifratura nasconde il contenuto del messaggio
- Non nasconde la dimensione del pacchetto cifrato
- La dimensione dipende ancora dal contenuto in chiaro

CRIME

Cos'è CRIME?

Compression Ratio Info-leak Made Easy

- Sfrutta la correlazione tra contenuto e dimensione del pacchetto
- Non viola la cifratura — attacca i metadati
- Ricostruisce il segreto un carattere alla volta

Prerequisiti

- Compressione TLS attiva sulla connessione
- Attaccante in grado di osservare il traffico cifrato
- Attaccante in grado di iniettare payload nel flusso

Esperimenti

Comprensione del segnale

- Ripetizione carattere → ZLIB non cresce linearmente
- Match/Mismatch progressivo → 1 byte di differenza rivela il carattere corretto
- Confronto sistematico → il segnale è deterministico e affidabile

↓

Analisi delle condizioni

- Lunghezza del segreto → non protegge dall'attacco
- Caratteri ripetuti → falsi positivi gestibili
- Mappa del segnale → visualizzazione completa della distribuzione

↓

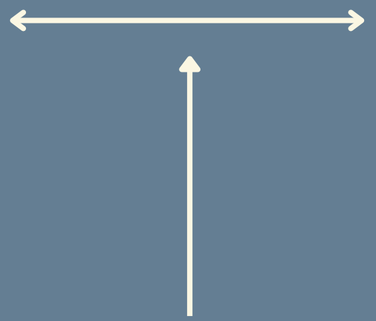
Attacco automatizzato

- Byte-by-byte → ricostruzione autonoma del segreto
- CRIME vs forza bruta → maggiore velocità ed efficienza

Mitm

Server

- Ha in memoria il segreto
- Comprime i dati prima di inviarli
- Non sa di essere sotto attacco



Client

- Invia richieste HTTP realistiche
- Comunica normalmente
- Non si accorge dell'attaccante

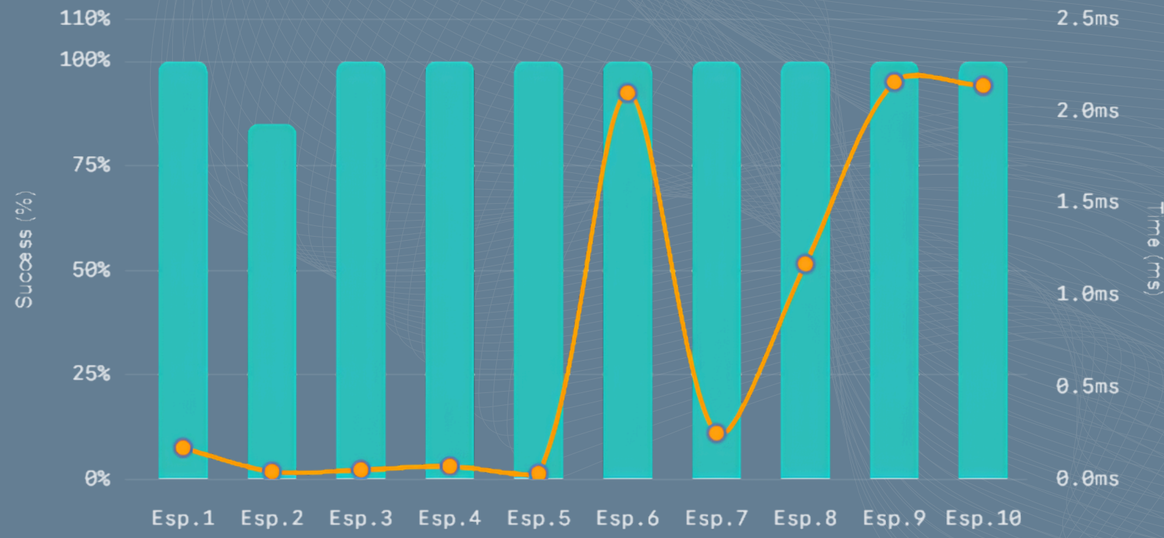
Attaccante (MitM)

- Si interpone tra client e server
- Osserva le dimensioni dei pacchetti
- Inietta payload silenziosamente
- Ricostruisce il segreto carattere per carattere

Risultati

Esperimento	Tentativi	Successo (%)	Tempo (ms)
Esp. 1 – Ripetizione carattere	6	100%	0,17
Esp. 2 – Match / Mismatch progressivo	7	85%	0,04
Esp. 3 – Confronto sistematico	8	100%	0,05
Esp. 4 – Lunghezza segreto	8	100%	0,07
Esp. 5 – Caratteri ripetuti	5	100%	0,03
Esp. 6 – Byte-by-byte automatizzato	520	100%	2,10
Esp. 7 – Mappa del segnale	62	100%	0,25
Esp. 8 – CRIME vs Forza Bruta	248	100%	1,17
Esp. 9 – MitM silenzioso	520	100%	2,16
Esp. 10 – HTTP simulato	520	100%	2,14

Tentativi totali: 1.904
Tempo totale: 8,17 ms
Percentuale media di successo: 98,0%



Conclusione e Sviluppi Futuri

Conclusioni

- Un sistema non è sicuro solo perché gli algoritmi sono robusti
- La vulnerabilità nasce dall'interazione inattesa tra compressione e cifratura
- 1 solo byte di differenza è sufficiente per compromettere un segreto
- L'unica contromisura efficace è la disabilitazione della compressione TLS

Sviluppi futuri

- Sessione HTTP completa con autenticazione e navigazione persistente
- Estensione a una rete virtuale reale eliminando i socket locali
- Analisi di vulnerabilità analoghe come BREACH e TIME



Università degli Studi di Milano

Grazie per l'attenzione